

Déployer et conduire

Une fois le matériel vérifié et les offsets enregistrés, intégrez YAGSL dans le code de votre robot. `SwerveParser` lit le répertoire de configuration de votre `swerve/base` et crée pour vous un YAMS `SwerveDrive` – vous n’avez plus qu’à écrire le sous-système et les éléments de contrôle autour de celui-ci.

Créer un sous-système swerve

```
import edu.wpi.first.math.geometry.Pose2d;
import edu.wpi.first.math.geometry.Rotation2d;
import edu.wpi.first.math.kinematics.ChassisSpeeds;
import edu.wpi.first.wpilibj.Filesystem;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
import java.io.File;
import java.util.function.DoubleSupplier;
import swervelib.parser.SwerveParser;
import yams.mechanisms.config.SwerveDriveConfig;
import yams.mechanisms.swerve.SwerveDrive;
import yams.mechanisms.swerve.utility.SwerveInputStream;
import yams.motorcontrollers.SmartMotorControllerConfig.TelemetryVerbosity;

public class SwerveDriveSubsystem extends SubsystemBase
{

    private SwerveDrive drive;

    public SwerveDriveSubsystem()
    {
        var cfg = new SwerveDriveConfig()
            .withStartingPose(new Pose2d(3, 3, Rotation2d.kZero))
            .withSubsystem(this)
            .withTelemetry(TelemetryVerbosity.HIGH);
        try
        {
            drive = new SwerveParser(new File(Filesystem.getDeployDirectory(),
"swerve/base"))
                .createSwerveDrive(cfg);
        } catch (Exception e)
        {

```

```

        throw new RuntimeException(e);
    }
}

public SwerveInputStream getAngularVelocityStream(DoubleSupplier x, DoubleSupplier
y,
                                DoubleSupplier rot)
{
    return new SwerveInputStream(drive, x, y, rot);
}

public Command drive(SwerveInputStream stream)
{
    return drive.drive(() -> ChassisSpeeds.fromFieldRelativeSpeeds(stream.get(),
                                                                    new
Rotation2d(drive.getGyroAngle())));
}

/** Mettre le cap du gyroscope à zéro. Associer cette commande à une combinaison de
boutons pour le recalibrage sur le terrain. */
public Command zeroGyro()
{
    return runOnce(() -> drive.zeroGyro());
}

@Override
public void periodic()
{
    drive.updateTelemetry();
}

@Override
public void simulationPeriodic()
{
    drive.simIterate();
}
}

```

`SwerveParser` lit `swervedrive.json` ainsi que chaque fichier auquel celui-ci fait référence dans `modules/` , identifie votre matériel grâce à la bibliothèque correspondante du fabricant et renvoie un YAMS `SwerveDrive` entièrement configuré – il n'y a pas de fichier de constantes à gérer manuellement.

`TelemetryVerbosity.HIGH` est utile lors de la mise en route du robot (ce sont les valeurs affichés sur le tableau de bord à l'étape 4), mais cela augmente la charge de `NetworkTables` – réduisez ce niveau une fois les réglages terminés.

Associer le contrôleur

```
import edu.wpi.first.wpilibj2.command.button.CommandXboxController;
import frc.robot.subsystems.swervedrive.SwerveDriveSubsystem;
import yams.mechanisms.swerve.utility.SwerveInputStream;

public class RobotContainer
{

    final CommandXboxController driverXbox = new CommandXboxController(0);

    private final SwerveDriveSubsystem swerve = new SwerveDriveSubsystem();

    private final SwerveInputStream driveAngularVelocity =
        swerve.getAngularVelocityStream(
            driverXbox::getLeftY,
            driverXbox::getLeftX,
            () -> driverXbox.getRawAxis(2))
            .withAllianceRelativeControl();

    public RobotContainer()
    {
        configureBindings();
    }

    private void configureBindings()
    {
        // Commande conduite par défaut
        swerve.setDefaultCommand(swerve.drive(driveAngularVelocity));

        // Mettre le gyroscope à zéro avec Start + Back – à utiliser si l'en-tête relatif
        // au terrain dérive
        driverXbox.start().and(driverXbox.back()).onTrue(swerve.zeroGyro());
    }
}
```

Pourquoi associer `zeroGyro()` à une combinaison de boutons? Les gyroscopes peuvent dériver ou s'allumer dans la mauvaise direction. Une combinaison de boutons (Start + Back, ou les deux bumpers) permet au pilote de réaligner instantanément le contrôle relatif du terrain sans toucher à la Driver Station. Toujours associer ceci – cela a déjà sauvé des matchs.

Première conduite

1. Déployez votre code
2. Avec le robot sur des blocs, activez-le en mode Teleop et poussez doucement le joystick de conduite vers l'avant. Les quatre roues devraient toutes être orientées dans la même direction et le robot doit essayer de se déplacer dans cette direction.
3. Faites tourner le robot avec l'axe de rotation. Il devrait tourner autour de son centre, sans dériver pendant la rotation.
4. Si un module ne fonctionne pas comme les autres (se place dans le mauvais angle ou avance en arrière par rapport aux autres), revoyez l'étape 4 pour vérifier son inversion/offset.
5. Une fois que le robot se déplace correctement sur les blocks, testez-le au sol à faible vitesse avant d'aller plus loin.

Félicitations – vous avez maintenant un robot swerve capable de se déplacer. À partir de là, consultez la section **How-to Guides** pour régler les gains PIDF, diagnostiquer les problèmes de dérive, et effectuer les autres tâches auxquelles vous reviendrez tout au long de la saison.